Versi 3.0

Agustus 2026

PEMROGRAMAN FUNGSIONAL

MODUL 3 - FIRST CLASS FUNCTION & DATA PROCESSING TECHNIQUES

Penulis:

FERA PUTRI AYU L., S.KOM., M.T.

HERDIANA DWI MAHARANI

NUR INDAH AMELIA AGUSTIN

LABORATORIUM INFORMATIKA

Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI **2**
PENDAHULUAN **3**
TUJUAN 3
TARGET MODUL 3
PERSIAPAN 3
KATA KUNCI 3
1. FIRST CLASS FUNCTION **4**
1.1. FUNGSI SEBAGAI VARIABEL 4
1.2. FUNGSI SEBAGAI PARAMETER 6
1.3. FUNGSI SEBAGAI RETURN VALUE 7
1.4. FUNGSI DI DALAM STRUKTUR DATA 9
2. HIGHER-ORDER FUNCTION **11**
2.1. BUILT-IN HIGHER-ORDER FUNCTIONS 12
3. PENGOLAHAN DATA DALAM PARADIGMA FUNGSIONAL **15**
3.1. FILTER 16
3.2. MAP 20
3.3. REDUCE 23
3.4. FUNGSI RECURSIVE **27**
CODE ANALYSIS **31**
ASSIGNMENT **32**
ASSIGNMENT 1 32
ASSIGNMENT 2 34
ASSESSMENT RUBRIC **35**
DETAILED ASSESSMENT RUBRIC 35

PENDAHULUAN

TUJUAN

1. Sub-CPMK 3: Mahasiswa mampu mengukur fleksibilitas suatu metode pemrograman fungsional terkait pemanfaatannya dalam perangkat lunak.

TARGET MODUL

1. Mahasiswa mampu menjelaskan dan menerapkan konsep First-Class Functions, termasuk menyimpan fungsi dalam variabel, meneruskan fungsi sebagai parameter, mengembalikan fungsi sebagai return value, serta menyimpan fungsi di dalam struktur data.
2. Mahasiswa mampu menjelaskan dan menerapkan Higher-Order Functions.
3. Mahasiswa mampu menerapkan teknik pemrosesan sequence data menggunakan HoF bawaan (filter(), map(), reduce()) untuk menyelesaikan kasus transformasi data dunia nyata, sesuai dengan prinsip immutability dan lazy evaluation.
4. Mahasiswa mampu menjelaskan dan mengimplementasikan fungsi rekursif sebagai alternatif loop imperatif dalam paradigma pemrograman fungsional.

PERSIAPAN

1. Bagi mahasiswa Muslim, disunnahkan membaca doa berikut sebelum memulai praktikum.

رَضِيْتُ بِاللهِ رَبًّا وَبِالإِسْلَامِ دِيْنًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُوْلًا. رَبِّ زِدْنِيْ عِلْمًا وَارْزُقْنِيْ فَهْمًا

2. Laptop atau komputer pribadi
3. Google Collab/VS Code/Jupyter Notebook
4. Source Code Modul 3
5. Motivasi dan komitmen yang kuat untuk belajar

KATA KUNCI

First Class Function, Higher-Order Function, Filter, Map, Reduce, Recursive Functions

1. FIRST CLASS FUNCTION

Dalam kehidupan sehari-hari, "first class" berarti punya hak istimewa, kayak naik pesawat kelas satu yang bikin kamu lebih bebas dan dapat akses yang lebih luas. Nah, dalam pemrograman, First-Class Function artinya fungsi diperlakukan sebagai warga kelas satu (first-class citizen), yang punya hak istimewa yang sama kayak tipe data lainnya (angka, string, objek).

Artinya, fungsi bisa kamu perlakukan layaknya data biasa: bisa disimpan dalam variabel, dikirim sebagai parameter, dikembalikan dari fungsi lain, sampai disimpan di dalam struktur data.

Hal ini bikin kode kamu jadi lebih fleksibel, ringkas, dan bisa dipakai berulang kali (reusable). Yuk, kita bahas satu per satu empat kemampuan yang dimiliki fungsi sebagai warga kelas satu ini.

1.1. FUNGSI SEBAGAI VARIABEL

Yuk, kita mulai dari kemampuan paling dasar dalam konsep First-Class Function, yaitu menyimpan fungsi ke dalam variabel. Biasanya, kalau dengar kata "variabel", yang langsung kepikiran itu angka, string, atau list. Tapi, dalam pemrograman fungsional (dan bahasa multi-paradigma kayak Python), fungsi juga bisa kamu simpan ke dalam variabel, lho. Bedanya, yang tersimpan di situ bukan hasil eksekusi fungsinya, melainkan referensi ke fungsi itu sendiri. Dengan kata lain, variabelnya nggak menyimpan nilai akhir, tapi lebih seperti "jalan pintas" menuju fungsi tersebut.

```python
# A simple function to check whether a number is positive
'''
pre: take any number 'n' as input
post: return the boolean result whether 'n' is positive or not
'''
def is_positive(n):
    return n > 0
```

```python
# Storing the is_positive function into the variable check_number

check_number1 = is_positive
check_number2 = is_positive(1)
```

Gambar 1.1. Fungsi Sebagai Variabel

📌 Exercise 1: Reference or Result?

Coba kamu cek gimana check_number1 dan check_number2 dibuat di cell sebelumnya. Sebelum kamu menjalankan kode baru, coba tebak dulu: mana yang merupakan referensi fungsi dan mana yang cuma nilai biasa?

📝 **Analysis Question:**

Sekarang, coba kamu buktikan tebakanmu lewat cell kode di bawah dengan memanggil check_number1(-5) dan check_number2(-5).

1. Pemanggilan mana yang berhasil dan mana yang muncul error?
2. Dari pesan *error*-nya tersebut, apa yang bisa kamu simpulkan terkait apa sebenarnya yang tersimpan di check_number2?
3. Jelaskan dengan bahasamu: apa bedanya menyimpan sebuah fungsi dengan menyimpan hasil pemanggilan fungsi tersebut?

```
# (Your NIM)
print(check_number1(-5))
#print(check_number2(-5))   # uncomment this line
```

```
False
```

Gambar 1.2. Exercise 1

Perbedaan Antara Fungsi Dengan dan Tanpa Tanda Kurung

Kenapa kadang kita menulis fungsi dengan tanda kurung, kadang tidak? Semua tergantung pada apa yang kamu mau simpan. Kalau tanpa tanda kurung (is_positive), yang tersimpan itu referensi ke fungsinya dan baru akan dieksekusi kalau kamu menambahkan tanda kurung. Sedangkan kalau pakai tanda kurung (is_positive(-5)), yang tersimpan adalah hasil eksekusinya: sebuah nilai, bukan fungsinya.

```
check_number1 = is_positive
check_number2 = is_positive
check_number3 = is_positive(-5)
check_number4 = is_positive(1)
```

```
print(check_number1(7))
print(check_number2)
```

```
True
<function is_positive at 0x7db718ba2ac0>
```

Gambar 1.3. Fungsi Dengan dan Tanpa Tanda Kurung

check_number1(7) memanggil fungsi tersebut dengan argumen baru (7), sehingga Python menjalankan is_positive(7) dan mengembalikan True. check_number2 dicetak tanpa tanda kurung, sehingga Python hanya menampilkan objek fungsinya sendiri (<function is_positive at ...>), bukan menjalankannya. Kedua variabel menyimpan hal yang sama,

yaitu referensi ke is_positive — bedanya hanya terletak pada apakah kita memanggilnya (dengan argumen yang wajib di dalam tanda kurung()) atau sekadar melihatnya saja.

```
print(check_number3)
print(check_number4(2))
```

```
False
---------------------------------------------------------------------------
-
TypeError                                 Traceback (most recent call
last)
/tmp/ipykernel_2049/2781238835.py in <cell line: 0>()
      1 print(check_number3)
----> 2 print(check_number4(2))

TypeError: 'bool' object is not callable
```

Gambar 1.4. Fungsi Dengan dan Tanpa Tanda Kurung

check_number3 dicetak tanpa tanda kurung, tetapi karena check_number3 = is_positive(-5) sudah dieksekusi, variabel ini menyimpan hasilnya, yaitu False, bukan sebuah fungsi — jadi yang tercetak adalah False. check_number4(2) menghasilkan error karena check_number4 = is_positive(1) juga sudah dieksekusi, sehingga menyimpan nilai boolean True, bukan sebuah fungsi, sedangkan menambahkan (2) berarti mencoba memanggil sebuah boolean seolah-olah itu fungsi, sehingga memunculkan TypeError: 'bool' object is not callable.

1.2. FUNGSI SEBAGAI PARAMETER

Setelah memahami bahwa fungsi dapat disimpan dalam variabel, langkah selanjutnya adalah mengirimkan fungsi tersebut sebagai argumen ke fungsi lain. Konsep ini mungkin terdengar sedikit abstrak, tetapi manfaatnya sangat besar: kamu bisa membuat fungsi "generik" yang perilakunya ditentukan oleh fungsi lain yang kamu berikan sebagai parameter. Konsep ini sering digunakan dalam pemrograman fungsional, misalnya pada fungsi bawaan Python seperti map, filter, dan reduce.

```python
# A few simple operations
def add(a, b):
    return a + b

def divide(a, b):
    return a / b if b != 0 else "Error: division by zero"

# A function that receives another function as a parameter
def operate(func, x, y):
    """Runs the given function with arguments x and y"""
    result = func(x, y)
    return result
```

```python
# Try sending different functions
print(operate(add, 10, 5))
print(operate(divide, 10, 2))
```

```
15
5.0
```

Gambar 1.5. Fungsi Sebagai Parameter

Dalam contoh ini, fungsi operate tidak mengetahui operasi apa yang harus dilakukan. Fungsi ini hanya mengetahui bahwa ia menerima sebuah fungsi func dan dua angka. Fungsi mana yang digunakan sepenuhnya bergantung pada argumen yang kamu berikan. Jika kamu memberikan add, maka akan dilakukan penjumlahan. Jika kamu memberikan divide, maka akan dilakukan pembagian. Dengan pendekatan ini, satu fungsi operate saja dapat melayani berbagai logika sekaligus tanpa perlu ditulis ulang. Inilah yang membuat kode lebih fleksibel dan lebih mudah digunakan kembali.

1.3. FUNGSI SEBAGAI RETURN VALUE

Dalam pemrograman prosedural, fungsi umumnya mengembalikan tipe data primitif (seperti integer, boolean) atau struktur data yang lebih kompleks (seperti array dan objek). Namun, dalam paradigma fungsional, karena fungsi merupakan warga kelas satu (first-class citizen), sebuah fungsi juga dapat menghitung dan mengembalikan fungsi-fungsi lainnya.

Sebuah fungsi juga bisa mengembalikan fungsi lain. Hal ini biasanya dilakukan dengan nested function (fungsi di dalam fungsi), karena nilai yang dikembalikan harus berasal dari komputasi internal fungsi itu sendiri, bukan dari konteks luar yang tidak terkendali. Konsep ini memungkinkan kamu menulis dynamic function factory—yaitu fungsi yang membangun dan menyesuaikan fungsi baru saat runtime berdasarkan argumen yang diterimanya.

Mengembalikan sebuah fungsi merupakan teknik fundamental dalam pemrograman fungsional yang membuka dua paradigma penting:

- **Closure & State Retention:** Fungsi yang dikembalikan (inner function) tetap dapat mengakses variabel yang didefinisikan dalam scope fungsi induknya (outer function), bahkan setelah fungsi induk tersebut selesai dieksekusi.
- **Currying & Partial Application:** Alih-alih menerima semua argumen sekaligus, sebuah fungsi dapat menerima satu argumen lalu mengembalikan fungsi baru yang menerima argumen-argumen sisanya, sehingga kode menjadi lebih modular dan dapat digunakan kembali.

Jangan khawatir kalau kedua konsep ini masih terasa abstrak. Kamu akan mempelajarinya lebih mendalam pada modul selanjutnya.

```
'''
desc: create_multiplier is a "function factory" -- it returns a new function
      instead of a plain value
pre-cond: factor is a number
post-cond: returns a function that multiplies its input by factor
'''
def create_multiplier(factor):
    def multiply(number):
        # 'multiply' remembers 'factor' from its enclosing scope (closure)
        return number * factor
    # return the inner function itself, NOT its result (no parentheses here)
    return multiply

# Create specialized functions dynamically
double = create_multiplier(2)   # a function that multiplies by 2
triple = create_multiplier(3)   # a function that multiplies by 3

# Execute the returned functions
print(double(5))   # 10
print(triple(5))   # 15

# We can also call the factory and the returned function in one line:
print(create_multiplier(10)(4))   # 40
```

```
10
15
40
```

Gambar 1.6. Function as a Return Value

Mari kita bedah apa yang terjadi di dalam create_multiplier:

1. ***Functions as values***: return multiply mengembalikan objek fungsinya sendiri, bukan hasil dari menjalankannya. Jika kamu menulis return multiply(), fungsi tersebut akan langsung dieksekusi dan itu bukan yang kita inginkan di sini.
2. ***State memory (closure)***: Saat kamu memanggil create_multiplier(2), parameter factor akan diset menjadi 2. Bahkan setelah create_multiplier selesai dieksekusi, fungsi yang

dikembalikan (double) tetap mengingat factor = 2 —ini yang dimaksud dengan closure.

3. ***Reusability***: Daripada menulis fungsi terpisah satu per satu seperti double(x), triple(x), atau quadruple(x) secara manual, satu fungsi factory dapat menghasilkan sebanyak apa pun fungsi multiplier khusus yang kamu butuhkan, sesuai permintaan.

1.4. FUNGSI DI DALAM STRUKTUR DATA

Setelah memahami bahwa fungsi dapat disimpan dalam variabel, tidak mengherankan jika fungsi juga dapat disimpan dalam struktur data seperti list, tuple, atau bahkan dictionary. Dengan cara ini, fungsi-fungsi dapat dikelompokkan layaknya kumpulan item yang dapat diakses melalui index atau key. Hal ini sangat berguna ketika kamu ingin mengelola banyak fungsi sekaligus, misalnya menyediakan beberapa jenis operasi dan memanggilnya sesuai kebutuhan pengguna. Mari kita lihat contoh berikut.

```
# A new HOF function for Make Power
def make_power(n):
    """Produces a function that raises x to the power of n"""
    def power(x):
        return x ** n
    return power
```

```
# Make sure you have run the cells above and the cell in section 1.2 also
# Storing various functions in a list
operations = [
    add,
    divide,
    make_power,
    make_power(2)
]

# Calling functions according to index order
print('add =', operations[0](10, 5))  # add(10, 5)
print('divide =', operations[1](10, 5))  # divide(10, 5)
print('power =', operations[2](2)(6))      # make_power(2)(6)
print('power of two=', operations[3](6))      # make_power(2)(6)
```

```
add = 15
divide = 2.0
power = 36
power of two= 36
```

Gambar 1.7. Fungsi di Dalam Struktur Data

Keempat elemen operasi tersebut berperilaku berbeda: tiga di antaranya menerima dua argumen, sedangkan yang terakhir, make_power(2) , hanya menerima satu argumen, karena nilainya sudah "dikunci" dengan n=2 saat closure-nya dibuat. Jadi, satu struktur

data bisa menampung fungsi-fungsi dengan logika yang berbeda, bahkan dengan jumlah argumen yang berbeda.

Selain list, kita juga bisa menyimpan fungsi dalam sebuah dictionary, dengan nama operasi sebagai key dan fungsinya sebagai value — berguna untuk kalkulator, router, atau command handler yang memilih logikanya berdasarkan input pengguna. Mari kita lanjutkan dengan fungsi add, multiply, dan make_power(2) yang sama seperti sebelumnya, kali ini disimpan dalam dictionary, bukan list.

```
# Using a dictionary to store functions
operation_dict = {
    "add": add,
    "divide": divide,
    "square": make_power(2)
}

print('add =', operation_dict["add"](7, 8))
print('divide =', operation_dict["divide"](3, 6))
print('square =', operation_dict["square"](9))
```

```
add = 15
divide = 0.5
square = 81
```

Gambar 1.8 Fungsi pada Dictionary

Pendekatan ini memberikan fleksibilitas yang besar: cukup memilih key dari dictionary, kamu dapat langsung menjalankan fungsi yang sesuai. Teknik semacam ini banyak digunakan dalam implementasi menu dinamis, command line interface, hingga framework yang mengelola alur logika berdasarkan event.

📌 Exercise 2: The Parentheses Trap

Coba kamu jalankan eksperimen di bawah ini (add() dan square() sudah didefinisikan pada cell sebelumnya). Dua elemen dari mixed berupa fungsi dan dua lainnya sudah berupa angka. Coba kamu cari tahu dulu mana yang mana sebelum kamu uncomment kodenya.

```
# (Your NIM)
mixed = [
    add,                # stored with or without parentheses?
    add(10, 5),
    make_power(2),
    make_power(2)(4)
]

#print(mixed)
```

```
#(Your NIM)

# Now try calling every element of the list by index
```

Gambar 1.9 Exercise 2

📝 **Analysis Question:**

Dalam list yang dicetak, dua elemen merepresentasikan sebuah fungsi, sedangkan dua lainnya menampilkan angka biasa. Jelaskan mana yang mana, dan mengapa peletakan tanda kurung menentukan hasilnya. Setelah memanggil setiap elemen list berdasarkan index-nya, analisis hasilnya. Apa yang kamu perlukan untuk mendapatkan hasil dari masing-masing elemen?

2. HIGHER-ORDER FUNCTION

Jika kamu sudah memahami First-Class Functions, maka Higher-Order Functions (HOF) adalah penerapan konkretnya. Sebuah fungsi disebut HOF jika ia dapat:

1. Menerima fungsi sebagai parameter (input), atau
2. Mengembalikan fungsi sebagai hasil (output).

Dengan kata lain, HOF memanfaatkan sifat first-class dari fungsi untuk menciptakan perilaku yang lebih dinamis, dapat digunakan kembali (reusable), dan lebih modular. Contoh sederhana dari HOF dapat kamu temukan pada fungsi seperti map(), filter(), atau reduce(), yang menerima fungsi lain untuk menentukan bagaimana data diproses.

📌 Exercise 3: The Swappable Engine

Pada cell sebelumnya, kamu sudah memiliki operations list (bagian 1.4) dan sudah memanggilnya berdasarkan index. Buat fungsi operasi baru untuk melakukan pengurangan, lalu tambahkan ke dalam operations list tanpa mengubah fungsi-fungsi yang sudah ada. Jalankan fungsi yang baru ditambahkan tersebut, lalu cetak hasilnya.

```
# (Your NIM)
# Write your own subtraction function,

# add it to the operations list

# and do the execution
```

Gambar 2.1. Exercise 3

📝 **Analysis Question:**

Jelaskan bagaimana desain higher-order memperlakukan fungsi sebagai nilai (value) yang dapat disimpan dalam sebuah list. Jika operations ditulis dengan cara "tradisional" yaitu satu fungsi tunggal dengan rangkaian if/elif yang memilih antara add, subtract, multiply, dan lain-lain bagaimana cara kamu memodifikasi fungsi tersebut agar mendukung semua operasi yang dibutuhkan? Mengapa versi HOF lebih mudah dikembangkan?

2.1. BUILT-IN HIGHER-ORDER FUNCTIONS

Python sendiri sudah menyediakan beberapa Higher-Order Functions bawaan, yaitu fungsi-fungsi yang siap pakai karena sudah dirancang untuk menerima fungsi lain sebagai argumen. Tiga di antaranya yang paling penting untuk memproses kumpulan data (data collections) adalah:

Aspek	map()	filter()	reduce()
Tujuan	Menerapkan fungsi transformasi ke setiap elemen dalam *iterable*	Menyaring elemen dalam *iterable*	Mereduksi *iterable* menjadi satu nilai menggunakan fungsi kumulatif
Modul	Built-in	Built-in	functools
Argumen Fungsi	Fungsi dengan satu argumen (satu elemen dari *iterable*)	Fungsi dengan satu argumen (satu elemen dari *iterable*)	Fungsi dengan dua argumen (akumulator dan elemen berikutnya)
Hasil Akhir	Map Object (lazy)	Filter Object (lazy)	Satu nilai tunggal
Iterasi	TRUE	TRUE	FALSE, hanya mengembalikan satu nilai
Panjang Hasil	Sama dengan panjang *iterable* asli	Bisa lebih pendek dari *iterable* asli	Satu nilai tunggal
Contoh Penggunaan	Menggandakan setiap angka dalam list	Menyaring angka genap/ganjil dari list	Menjumlahkan semua angka dalam list

Gambar 2.2. Built-in Higher-Order Functions

Ketiga fungsi tersebut merupakan bukti konkret bahwa fungsi adalah first-class citizen di Python: masing-masing menerima fungsi lain sebagai input, persis seperti contoh calculate() yang sudah kita bahas sebelumnya. Kita akan menggunakan fungsi bernama biasa (yang didefinisikan dengan def, bukan lambda) sebagai "logika" yang kita berikan ke fungsi-fungsi tersebut, supaya kode tetap eksplisit dan mudah ditelusuri.

Di sini kita hanya membahas signature dan mekanismenya secara sekilas. Implementasi lengkap beserta contoh langkah demi langkah pada data numerik, dictionary, string, dan nested list akan dibahas secara mendetail pada **Bagian 3: Teknik Pemrosesan Data dalam Paradigma Fungsional**, tepat setelah kita membahas mengapa immutability penting untuk jenis transformasi data seperti ini.

Mari kita lihat ketiganya dalam contoh minimal untuk masing-masing, supaya konsepnya jadi lebih konkret sebelum kita lanjut lebih jauh. Kita akan menggunakan sebuah list angka sederhana beserta tiga fungsi bernama sederhana:

```
numbers = [1, 2, 3, 4, 5]

# a one-argument function: doubles a value
def double(x):
    return x * 2

# a one-argument function: True when x is even
def is_even(x):
    return x % 2 == 0

# a two-argument function: adds two values
def add(a, b):
    return a + b

from functools import reduce

print(list(map(double, numbers)))      # map    -> transform every element
print(list(filter(is_even, numbers))) # filter -> keep only the elements tha
print(reduce(add, numbers))           # reduce -> fold every element into on
```

```
[2, 4, 6, 8, 10]
[2, 4]
15
```

Gambar 2.3. Contoh Built-in Higher-Order Functions

Tiga baris kode, tiga tugas yang berbeda:

- map(double, numbers) mengirim setiap elemen ke double, sehingga menghasilkan sequence baru [2, 4, 6, 8, 10] .
- filter(is_even, numbers) mengirim setiap elemen ke is_even; hanya elemen yang menghasilkan True yang lolos, sehingga menghasilkan [2, 4].

- reduce(add, numbers) terus menggabungkan pasangan nilai menggunakan add — (((1+2)+3)+4)+5 — sampai tersisa satu nilai, yaitu 15.

Pada setiap kasus, kamu hanya perlu menyediakan fungsi bernama sederhana (double, is_even, add); sementara map/filter/reduce yang menangani proses perulangannya. Pemisahan tugas inilah yang membuat ketiganya disebut sebagai Higher-Order Functions. Ini baru sekadar pratinjau singkat untuk membuat idenya lebih konkret — implementasi lengkapnya, termasuk lazy evaluation, dictionary, string, nested list, dan initializer pada reduce, akan dibahas secara mendalam pada **Bagian 3**.

Bagaimana cara kerja masing-masing fungsi ini, langkah demi langkah?
map(double, numbers) menelusuri numbers satu elemen demi satu elemen dan memanggil double pada setiap elemennya:

1. double(1) → 2
2. double(2) → 4
3. double(3) → 6
4. double(4) → 8
5. double(5) → 10

Hasil: sequence baru [2, 4, 6, 8, 10] — panjangnya sama seperti input, namun nilainya sudah ditransformasi.

filter(is_even, numbers) juga mengunjungi setiap elemen, tetapi alih-alih mentransformasinya, fungsi ini mengajukan pertanyaan ya/tidak kepada is_even dan hanya mempertahankan elemen yang jawabannya True:

1. is_even(1) → False → dropped
2. is_even(2) → True → kept
3. is_even(3) → False → dropped
4. is_even(4) → True → kept
5. is_even(5) → False → dropped

Hasil: [2, 4] — jumlah elemennya sama atau lebih sedikit, sementara nilai elemennya tidak berubah.

reduce(add, numbers) berbeda lagi; ia sama sekali tidak membentuk sebuah sequence. Fungsi ini membawa sebuah akumulator, menggabungkan dua nilai sekaligus setiap kali sampai hanya tersisa satu nilai saja:

1. add(1, 2) → 3
2. add(3, 3) → 6
3. add(6, 4) → 10
4. add(10, 5) → 15

Hasil: satu nilai tunggal, yaitu 15.

Perhatikan pola pada signature fungsi-fungsi tersebut: map dan filter sama-sama menerima fungsi dengan satu argumen dan mengembalikan sebuah sequence, sedangkan

reduce menerima fungsi dengan dua argumen dan mengembalikan satu nilai. Perbedaan ini bukan suatu kebetulan. Perbedaan ini muncul langsung dari apa yang dirancang untuk dilakukan oleh masing-masing fungsi: mengubah setiap elemen (map), menyaring sebagian (filter), atau melipatnya menjadi satu (reduce).

📌 Exercise 4: The Truthy Trap

Di atas, is_even(x) mengembalikan x % 2 == 0, sehingga filter(is_even, numbers) hanya menyimpan nilai-nilai yang hasilnya True. Sekarang, buatlah versi kedua yang mengembalikan x % 2, lalu jalankan melalui filter() dengan cara yang sama.

```
#(Your NIM)
def _____(x):
    return _____

print(list(filter(is_even, numbers)))
print(list(filter(_____, numbers)))
```

Gambar 2.4. Exercise 4

📝 Analysis Question:

Bandingkan kedua hasilnya. Apakah filter() berperilaku sama pada keduanya? Apa yang bisa kamu simpulkan dari hal ini tentang apa yang sebenarnya dibutuhkan filter() pada fungsi yang diberikan? Apakah filter() memerlukan nilai boolean yang sesungguhnya, atau cukup sesuatu yang dianggap truthy/falsy oleh Python?

3. PENGOLAHAN DATA DALAM PARADIGMA FUNGSIONAL

Dalam paradigma fungsional, program yang kita buat merupakan rangkaian transformasi data dari satu bentuk ke bentuk lain. Mari kita lihat beberapa contoh nyata bagaimana paradigma fungsional ini bekerja di bidang kita (informatika):

1. Pada pemrograman back-end server, kita dapat memodelkannya sebagai proses yang mentransformasikan HTTP request menjadi HTTP response.
2. Pada pemrograman front-end, kita dapat memodelkannya sebagai proses yang mentransformasikan data dari server menjadi representasi UI.
3. Sebuah compiler program yang mentransformasikan source code menjadi executable.

Transformasi data mengandalkan prinsip immutability: sekali data dibuat, data tersebut tidak akan diubah. Setiap fungsi mengembalikan nilai baru, bukannya memutasi (mengubah) data aslinya, sehingga data tetap dapat diprediksi dan hanya dapat berubah melalui fungsi transformasi yang kita buat. Konsep ini sudah kita temukan pada Modul 2 sebagai salah satu syarat pure function. Inilah ciri ketiga dari paradigma fungsional: mentransformasi data, menerapkan immutability, dan tidak pernah memutasi data.

Ketika kamu bekerja dengan kumpulan data (koleksi) seperti range, list, tuple, dictionary, dan sebagainya, ada dua pola yang sangat umum digunakan:

1. Melakukan iterasi untuk membangun koleksi baru, mentransformasi atau menguji setiap item, lalu menambahkan hasilnya ke koleksi baru (inilah ide dasar dari map() dan filter()).
2. Melakukan iterasi dan mengakumulasi menjadi satu nilai — misalnya seperti len(), min(), max(), sum(), dan reduce().

Dalam praktiknya, kita sebaiknya menghindari penulisan for/while loop secara imperatif untuk kedua pola tersebut dan lebih memilih menggunakan Higher-Order Functions bawaan Python, yaitu filter(), map(), dan reduce(), yang signature-nya sudah kita bahas pada Bagian 2. Dikombinasikan dengan immutability dan lazy evaluation, ketiga fungsi ini memungkinkan kamu mengekspresikan transformasi yang kamu inginkan, bukan bagaimana cara melakukan perulangan langkah demi langkah — sehingga kode tetap ringkas, deklaratif, dan bebas dari efek samping (side effects).

Mari kita bahas ketiganya secara lebih mendalam, lengkap dengan kasus-kasus nyata.

3.1. FILTER

List comprehension dengan if juga dapat melakukan filtering, tetapi menghitung semuanya sekaligus (eager evaluation). Fungsi filter() bawaan Python lebih efisien karena bersifat lazy, artinya data baru diproses saat benar-benar dibutuhkan.

filter () menerima dua argumen — (function, iterable):

1. Fungsi logika → mengembalikan True/False untuk setiap elemen.
2. Sebuah iterable → data yang akan di filter.

Filter akan menjalankan fungsi logika tersebut pada setiap elemen; True akan menyimpan elemen tersebut, sedangkan False akan membuangnya. Hasilnya berupa sebuah filter object yang baru dievaluasi saat benar-benar dibutuhkan. Detail lebih lanjut geeksforgeeks or realython.

Contoh 1: Memfilter Data Numerik

```
# first, prepare a logic function
def passed(score):
  ''' function to check whether a score passes
      pre-cond: score is a number
      post-cond: Boolean value; True if score >= 70
  '''
  return score >= 70

# Student score data
scores = [55, 80, 72, 65, 90, 45, 88, 60, 90, 50, 73]
```

```
# using filter
passing_scores = filter(passed, scores)

print(passing_scores)
```

```
<filter object at 0x7b06e0ced090>
```

Gambar 3.1. Memfilter Data Numerik

Hasil di atas berupa sebuah filter object karena sifatnya yang lazy. Untuk melihat isinya, kamu bisa mengaksesnya dengan next(), menggunakan for loop, atau langsung mengonversinya ke tipe data lain (list/tuple) melalui casting.

```
# Accessing the filter result above with next()
print('First item:', next(passing_scores))
print('Second item:', next(passing_scores))

# Accessing the remaining items with a loop
print('The rest:')
for item in passing_scores:
    print(item, end=" ")
```

```
First item: 80
Second item: 72
The rest:
90 88 90 73
```

Gambar 3.2. Akses Filter

Konversi menjadi data tuple:

```
# the filter object needs to be recreated
# because its elements have already been exhausted from the previous iteratic
passing_scores = filter(passed, scores)
list(passing_scores)
```

```
[80, 72, 90, 88, 90, 73]
```

Gambar 3.3. Konversi ke Tuple

Contoh 2: Memfilter Data Dictionary (Mahasiswa)

```
students = [
    {"name": "Dika", "score": 100},
    {"name": "Wira", "score": 55},
    {"name": "Fatih", "score": 85},
    {"name": "Alif", "score": 45},
    {"name": "Ega", "score": 90}
]

# logic function for student data
'''
 function to check whether a student passes based on their score
 pre-cond: score is a dictionary that has a "score" key inside it
 post-cond: True value if score >= 70
'''
def student_passed(score):
    return score["score"] >= 70
```

```
passing_students = filter(student_passed, students)
print(passing_students) #lazy
```

```
<filter object at 0x7a492f185d80>
```

Gambar 3.4. Memfilter Data Dictionary

```
print(tuple(passing_students)) #directly cast to a tuple type
```

```
({'name': 'Dika', 'score': 100}, {'name': 'Fatih', 'score': 85}, {'name': 'Ega', 'score': 90})
```

Gambar 3.5. Casting ke Tuple

📌 Exercise 5: The Empty Shelf

Diberikan data produk berikut. Lengkapi potongan kode pada cell berikutnya agar hanya menampilkan produk yang stoknya kosong (0).

```
products = [
    {"name": "Laptop", "stock": 3},
    {"name": "Mouse", "stock": 0},
    {"name": "Keyboard", "stock": 8},
    {"name": "Monitor", "stock": 0},
    {"name": "Flashdisk", "stock": 10}
]
```

Gambar 3.6. Product Data

Lengkapi bagian yang kosong (____) pada cell kode di bawah ini.

1. Sebuah fungsi logika yang memeriksa apakah stok suatu produk bernilai nol,
2. Sebuah pemanggilan filter() yang menggunakan fungsi tersebut
3. Tampilkan melalui iterasi (for loop — bukan hanya dengan casting ke tuple).

```
# (Your NIM)

def ____(product):
    """Return True when this product's stock is 0."""
    return product["____"]  #condition

out_of_stock = filter(____, products)
print(____(out_of_stock))
```

```
#(Your NIM)
for ____ in ____:
    print(____)
```

Gambar 3.7. Exercise 5

📝 **Analysis Question:**

Hasil filter() kamu, yaitu out_of_stock, merupakan sebuah lazy object. Apa yang terjadi jika kamu melakukan iterasi menggunakan for loop? Apa yang kamu dapatkan dan mengapa hal itu bisa terjadi?

3.2. MAP

Contoh 1: Mengonversi Nilai Numerik menjadi Huruf (Mapping)

```
# make sure the filter code cell has already been run
# Transformation function to be used inside the map() function
'''
 function to transform a score value into a letter grade
 pre-cond: score is a number
 post-cond: a letter grade of 'A', 'B', or 'C'
'''
def to_grade(score):
    if score >= 85:
        return "A"
    elif score >= 70:
        return "B"
    else:
        return "C"
```

```
#using map
grades = map(to_grade, scores)
print(scores)
print(grades)          # only displays the object (because it's lazy)
print(list(grades))    # cast/convert to a list to print all the results
```

```
[55, 80, 72, 65, 90, 45, 88, 60, 90, 50, 73]
<map object at 0x7b06e0ced6c0>
['C', 'B', 'B', 'C', 'A', 'C', 'A', 'C', 'A', 'C', 'B']
```

Gambar 3.8. Mapping Nilai Numerik menjadi Huruf

Sama seperti filter(), hasil map() juga bersifat lazy. Jika dicetak langsung, hanya akan menampilkan map object saja; untuk melihat isinya, kamu perlu mengonversinya ke list, menggunakan next(), atau melakukan iterasi dengan loop.

📌 Exercise 6: The Vanishing Iterator

Sebelum kamu menjalankan cell di bawah ini, coba telusuri kodenya **secara manual**: untuk setiap iterasi loop, tulis score dan nilai yang seharusnya dihasilkan oleh next(grades). Setelah itu, jalankan kodenya dan periksa hasil trace-mu.

```
grades = map(to_grade, scores)

print("Scores with Grades:")
# iterate to display the grades data using next()
for score in scores:
    print(score,'=',next(grades))

# Exercise 5 attempt — add your hand-trace as comments here, e.g.:
# score=55 -> next(grades) should be 'C'
# score=80 -> next(grades) should be 'B'
# ...
```

Gambar 3.9. Exercise 6

📝 **Analysis Question:**

Loop di atas menulis for score in scores, bukan for score in grades, meskipun yang kita cetak adalah grades. Mengapa melakukan iterasi terhadap scores sambil mengambil nilai dari grades menggunakan next() tetap bisa bekerja dengan benar? Apakah ada risiko terjadinya error StopIteration di sini?

Contoh 2: Mapping pada String

Bagaimana dengan data non-numerik seperti String? map() juga bisa diterapkan pada data tersebut, selama kedua persyaratannya terpenuhi: (1) sebuah fungsi transformasi dan (2) data berupa sequence/iterable. Sebuah string merupakan sequence karakter, sehingga bisa dijadikan input untuk map(). Mari kita buktikan:

```
#prepare a string transformation function
'''
desc: the uppercase(s) function converts the input string (s) into uppercase
pre-cond: input s must be of string type
post-cond: the input string is returned in uppercase form
'''
def uppercase(s):
  return s.upper()
```

```
string = 'myname'
caps_name = map(uppercase,string)
print(list(caps_name))
```

```
['M', 'Y', 'N', 'A', 'M', 'E']
```

Gambar 3.10. Mapping pada String

Contoh 3: Mapping pada List of String

```
# List of strings
name_list = ['wendy', 'ian', 'ayu']

# Function to append an email format to a name
def add_email(name):
    return name + "@gmail.com"

emails = map(add_email, name_list)
print(list(emails))
```

```
['wendy@gmail.com', 'ian@gmail.com', 'ayu@gmail.com']
```

```
#using the uppercase() function on name_list
caps_name = map(uppercase,name_list)
print(list(caps_name))
```

```
['WENDY', 'IAN', 'AYU']
```

Gambar 3.11. Mapping pada List of String

Apa yang akan terjadi jika fungsi uppercase() digunakan pada data scores, yang bertipe numerik?

```
scores = [80, 75, 90]

caps_name = map(uppercase,scores)
print(list(caps_name)) #will error
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
/tmp/ipykernel_604/525669458.py in <cell line: 0>()
      2
      3 caps_name = map(uppercase,scores)
----> 4 print(list(caps_name)) #will error

/tmp/ipykernel_604/3628370157.py in uppercase(s)
      6 '''
      7 def uppercase(s):
----> 8   return s.upper()

AttributeError: 'int' object has no attribute 'upper'
```

Langkah berikutnya: Jelaskan error

Gambar 3.12. Uppercase Test

"Lhooo, kok bisa error??"
Karena fungsi upper() hanya bisa dijalankan pada string, bukan pada integer. Error ini terjadi karena input yang diberikan tidak memenuhi precondition dari fungsi tersebut. Artinya, fungsi uppercase() mengharapkan data berupa string, sementara yang diberikan justru berupa angka.

Contoh 4: Mapping pada Nested List

Setelah mempelajari konsep dasar fungsi map(), yang memungkinkan kita menerapkan sebuah fungsi ke setiap elemen dalam list secara efisien, kita bisa melangkah lebih jauh dengan menerapkannya pada nested list (list bersarang). Fungsi map() juga bisa digunakan untuk menerapkan sebuah fungsi pada setiap elemen dalam nested list secara fungsional.

```
# Student score data per subject
student_scores = [
    [80, 75, 90],   # Student 1
    [60, 70, 65],   # Student 2
    [85, 88, 92]    # Student 3
]

# Function to add a +5 bonus to a score
def bonus(score):
    return score + 5
```

Gambar 3.13. Map pada Nested List

📌 Exercise 7: Mapping Over Rows

Kode di bawah ini mencoba menerapkan bonus() secara langsung pada nested list student_scores, kemudian menerapkannya dengan benar baris per baris (row by row) menggunakan list comprehension. Jalankan kode di bawah ini untuk memeriksa hasilnya!

```
list(map(bonus,student_scores))
```

```
# Apply map to each row (each student's scores)
bonus_scores = [list(map(bonus, row)) for row in student_scores]

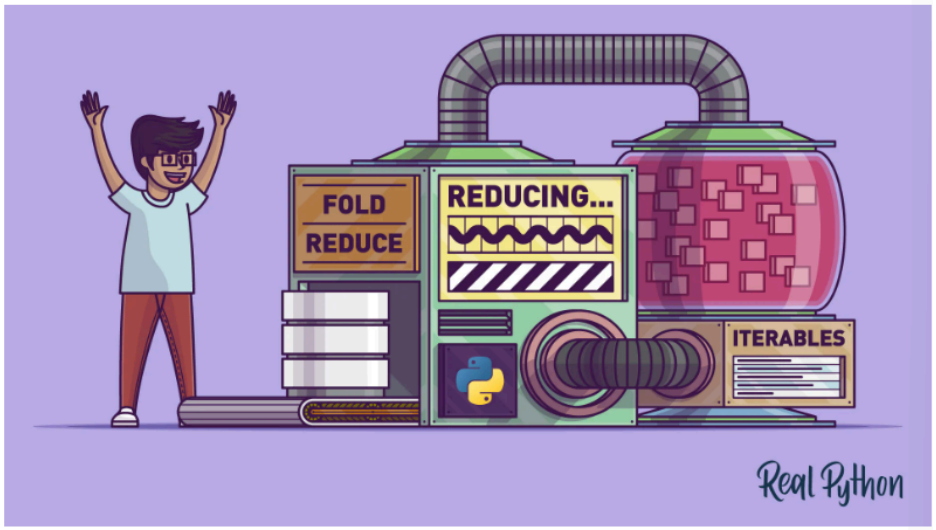
print("original scores\t\t:", student_scores)
print("Total scores + bonus\t:", bonus_scores)
```

Figure 3.14. Exercise 7

📝 Analysis Question:

Jelaskan mengapa list(map(bonus, student_scores)) tidak bekerja sesuai harapan ketika diterapkan langsung pada nested list, dan mengapa perlu melakukan looping pada setiap baris (row) menggunakan list comprehension sebelum memanggil map() agar mendapatkan hasil yang benar.

3.3. REDUCE



Fungsi reduce() dalam Python digunakan untuk meringkas sebuah kumpulan data menjadi hanya satu nilai saja. Perbedaannya dengan dua fungsi sebelumnya:

1. map() mentransformasi setiap elemen,
2. filter() menyaring sebagian elemen,

Sedangkan reduce() menggabungkan seluruh elemen hingga tersisa satu hasil akhir. Fungsi ini membutuhkan setidaknya dua argumen, yaitu:

1. **Sebuah fungsi dengan dua argumen** (fungsi yang menerima 2 input dan menghasilkan 1 output)
2. **Iterable** (data sequence seperti list atau tuple).

Selain itu, ada juga initializer, yaitu argumen ketiga yang bersifat opsional, yang akan dibahas lebih lanjut nanti.

Contoh: Fungsi Sederhana Untuk Penjumlahan

```
'''
desc: Function to add a and b
pre-cond: a and b can be any data
post-cond: a+b as the return value
'''
def add(a, b):
  result =  a + b
  return result

add(10, 7)
```

```
17
```

Gambar 3.15. Fungsi Sederhana Untuk Penjumlahan

Fungsi add() cocok digunakan di dalam reduce() karena memenuhi persyaratannya: menerima dua argumen (a dan b) lalu mengembalikan satu nilai hasil.

Sebelum menggunakan fungsi reduce, kita perlu meng-import modul/library tempat reduce berada. reduce() bukan merupakan fungsi built-in Python secara langsung, melainkan berada di dalam modul functools. Jadi, sebelum menggunakannya, kita harus melakukan import dari library functools terlebih dahulu.

```
from functools import reduce
```

Gambar 3.16. Import Library

Eksperimen 1: Reduce pada List Angka

```
scores = [50, 70, 90]

# Calculate the total score with reduce
total = reduce(add, scores)
print("Total score:", total)
```

```
Total score: 210
```

Gambar 3.17. Reduce pada List Angka

Proses di atas sama dengan menjalankan operasi ((50 + 70) + 90) = 210. Jadi, reduce() melipat elemen-elemennya selangkah demi selangkah hingga hanya tersisa satu nilai.

reduce() juga menerima argumen ketiga yang bersifat opsional, yaitu initializer — titik awal dari proses reduksi. Tanpa initializer, reduce akan dimulai dari dua elemen pertama; dengan initializer, reduce akan dimulai dari initializer dan elemen pertama (seolah-olah ada elemen tambahan yang diletakkan di depan). Contoh:

```
#adding the value 100 as the initializer/3rd argument of reduce
total_bonus = reduce(add, scores, 100)
print("Total score with bonus:", total_bonus)
```

```
Total score with bonus: 310
```

Figure 3.18. Reduce Initializer

Nilai [50, 70, 90] dijumlahkan dimulai dari 100:

1. 100 + 50 = 150
2. 150 + 70 = 220
3. 220 + 90 = 310

Hasil akhir: 310. Initializer sangat berguna untuk menentukan nilai awal. Seperti poin bonus, saldo awal, offset, atau nilai default lainnya yang dihitung sejak awal.

Eksperimen 2: Reduce pada Hasil map()

```
caps_string = map(uppercase,string)
list(caps_string)
```

```
['M', 'Y', 'N', 'A', 'M', 'E']
```

Gambar 3.19. Reduce pada hasil map()

Bagaimana cara menggabungkan kembali karakter-karakter yang dihasilkan oleh map() menjadi satu string? Kamu bisa menggunakan reduce() bersama add() untuk menggabungkannya. Perlu diingat: operator + di dalam add() juga bekerja pada string (sebagai concatenation). Ingat kembali precondition-nya yang memperbolehkan tipe data apa pun. Mari kita buktikan:

```
caps_string = reduce(add,map(uppercase,string))
caps_string
```

```
'MYNAME'
```

Gambar 3.20. Reduce Mapping

Eksperimen 3: Reduce pada Nested List

```
# the student_scores data is already available from the map-nested list matei
print("Original data:", student_scores)
```

```
Original data: [[80, 75, 90], [60, 70, 65], [85, 88, 92]]
```

Gambar 3.21. Reduce pada Nested List.

Sekarang, coba kita "lipat" nested list ini menggunakan reduce(), sama seperti yang sudah kita lakukan sebelumnya pada flat list angka. Karena student_scores merupakan list yang berisi list-list lainnya (satu list untuk setiap mahasiswa), maka elemen yang diiterasi oleh reduce() bukanlah angka satuan, melainkan sublist secara utuh. Hal ini akan memengaruhi apa yang sebenarnya diterima pada setiap pemanggilan add(a, b), seperti yang akan kamu lihat pada latihan di bawah ini.

📌 **Exercise 8: Flatten & Sum**

Pastikan kamu sudah menjalankan semua cell sebelumnya (latihan ini menggunakan student_scores dan add()).

a. Cell di bawah ini masih salah. Jalankan cell tersebut dan amati hasilnya sebelum melanjutkan membaca.

```
reduce1 = reduce(add, student_scores)
# (Your NIM)
# Exercise 8a
```

Gambar 3.22. Exercise 8

b. Sekarang coba kamu perbaiki: tulis kode yang menjumlahkan semua angka di dalam student_scores menjadi satu grand total. (Petunjuk: ratakan (flatten) dulu nested list-nya menggunakan loop, atau terapkan reduce dua kali — sekali untuk setiap baris, lalu sekali lagi untuk menjumlahkan total tiap baris.)

📝 **Analysis Question:**

Mengapa reduce(add, student_scores) biasa pada bagian (a) tidak bisa melipat (collapse) menjadi satu angka seperti yang terjadi pada flat list of numbers? Dengan melihat apa yang sebenarnya diterima oleh add(a, b) pada pemanggilan pertamanya, apa yang bisa kamu simpulkan tentang bagaimana reduce memperlakukan setiap elemen dari iterable yang diberikan kepadanya — apakah reduce "melihat" angka-angka di dalam sub-list, atau hanya sub-list itu sendiri? Dan jelaskan bagaimana jawabanmu pada bagian (b) dapat menyelesaikan masalah ini!

3.4. FUNGSI RECURSIVE

Kamu sudah melihat bagaimana map(), filter(), dan reduce() memungkinkan kita mengubah (transform), memilih (select), dan mengumpulkan (aggregate) sequence tanpa harus menulis for atau while loop secara manual. Tool-tool ini mengabstraksikan proses iterasi dengan berfokus pada apa yang harus terjadi pada setiap elemen, bukan bagaimana cara melangkah melalui index-nya.

Namun, masih ada satu pertanyaan mendasar:
"Bagaimana kita melakukan repetisi ketika abstraksi sequence bawaan tidak cukup, atau ketika memproses struktur data custom?"

Dalam pemrograman imperatif, kamu mengandalkan mutable loop counter (i += 1) dan pembaruan state secara in place. Dalam pemrograman fungsional murni, repetisi diekspresikan melalui recursion (rekursi): memecah sebuah masalah menjadi instance-instance yang lebih kecil dari dirinya sendiri hingga mencapai base case.

Bahkan, map, filter, dan reduce sebenarnya bukan sihir; lingkungan fungsional sering kali mendefinisikan abstraksi pemrosesan data tersebut menggunakan logika rekursif murni.

Sebuah fungsi rekursif memanggil dirinya sendiri selama proses eksekusi. Fungsi ini cocok untuk masalah-masalah yang bisa dipecah menjadi submasalah yang lebih kecil dengan bentuk yang sama (factorial, Fibonacci, deret geometri, …), yang sering kali menghasilkan kode yang lebih singkat dan jelas dibandingkan dengan loop.

Contoh dasar: factorial
Faktorial dari n adalah hasil perkalian semua bilangan bulat dari 1 hingga n,
contoh 5! = 5 × 4 × 3 × 2 × 1 = 120.

```
'''
desc: function to recursively calculate the factorial of a number n
pre-cond: input n is an integer >= 0
post-cond: the factorial value of n
'''
def recursive_factorial(n):
    if n <= 1:
        return 1  # Base Case
    else:
        return n * recursive_factorial(n - 1)  # Recursive Case

print(recursive_factorial(5))
```

```
120
```

Gambar 3.23. Fungsi Recursive

Cara kerjanya:

- factorial(5) → 5 × factorial(4)
- factorial(4) → 4 × factorial(3)
- factorial(3) → 3 × factorial(2)
- factorial(2) → 2 × factorial(1)
- factorial(1) → 1 (base case — rekursi berhenti dan mengembalikan nilai)

Jadi, prosesnya adalah 1 × 2 × 3 × 4 × 5 = 120. Nilai inilah yang dikembalikan dan dicetak.

Sebuah fungsi rekursif terdiri dari dua bagian:

1. **Base Case** — kondisi yang menghentikan rekursi. Tanpa base case, fungsi akan terus memanggil dirinya sendiri tanpa henti dan menyebabkan stack overflow.
2. **Recursive Case** — bagian yang memanggil dirinya sendiri pada submasalah yang lebih kecil, hingga akhirnya mencapai base case.

Anggaplah rekursi sebagai fondasi dari reduce: di setiap langkah, kamu mengambil elemen saat ini, memprosesnya, lalu meneruskan hasil yang sedang berjalan (running result) ke langkah berikutnya. Perbedaannya: reduce memanggil fungsi lain, sedangkan rekursi memanggil dirinya sendiri. Factorial sebelumnya juga bisa ditulis menggunakan reduce:

```
def multiply(a, b):
    return a * b

print(reduce(multiply, range(1, 6))) # multiplies every number in the range
```

```
120
```

Gambar 3.24. Reduce

Contoh Lainnya: Menghitung Panjang Data

Selain faktorial, kita juga bisa menggunakan rekursi untuk menghitung panjang suatu data. Meskipun Python sudah menyediakan fungsi len(), versi rekursif ini berguna untuk memahami konsep rekursi secara lebih mendalam.

```
# let's use the data from the result of Experiment 2 in the reduce material,
caps_string
```

```
'MYNAME'
```

```
# without recursion
def count_loop(data):
    count = 0
    for char in data:
        count += 1
    return count

print("Length of data",caps_string,'=', count_loop(caps_string))
```

```
Length of data MYNAME = 6
```

```
# using recursion
def count_recursive(data):
    if data == '': # Base Case: if the string is empty
        return 0
    else:  # Recursive Case: 1 + count_recursive(data--starting from index 1
        return 1 + count_recursive(data[1:]) #excluding the element at index

print("String length (recursive):", count_recursive(caps_string))  #Output: (
```

```
String length (recursive): 6
```

Gambar 3.25. Recursive Count

Decomposing Data: Head and Tail

Memproses sebuah sequence secara rekursif bergantung pada pola sederhana: memecah sequence menjadi head (elemen pertama) dan tail (seluruh elemen yang tersisa).

```
data = 'MYNAME'

head = data[0]  #'M'
tail = data[1:] #'YNAME'
```

Figure 3.26. Decomposing Data

Rekursi cocok dengan paradigma fungsional karena rekursi menggantikan loop imperatif (for/while) dan menghindari side effect seperti counter yang terus berubah. Rekursi secara alami cocok untuk masalah-masalah yang bisa dipecah menjadi submasalah yang lebih kecil dan serupa, seperti menghitung jumlah karakter dalam sebuah string: ambil satu karakter, lalu hitung sisanya dengan cara yang sama, berulang kali.

📌 Exercise 9: Think Recursively

a. Lengkapi fungsi rekursif sum_list(data) di bawah ini agar dapat menjumlahkan semua angka dalam flat list, tanpa menggunakan sum(), reduce(), atau loop. Tentukan base case dan recursive case-mu dalam bentuk komentar.
b. Gunakan fungsi tersebut untuk menghitung total dari student_scores yang sudah di-flatten, lalu periksa apakah hasil rekursif sama dengan hasil dari reduce pada Latihan 8.

```
#(Your NIM)

def sum_list(data):
    # base case: empty list -> 0
    # recursive case: first element/head + sum_list(the rest/tail)
    pass
```

Gambar 3.27. Exercise 9

📝 Analysis Question:

Reduce dan rekursi sama-sama menggabungkan banyak nilai menjadi satu nilai. Pada sum_list milikmu, bagian mana yang berperan sama seperti *accumulator* pada reduce? Tunjukkan *base case*-mu secara eksplisit dan jelaskan mengapa fungsi tersebut tidak akan pernah berhenti (*terminate*) tanpa adanya *base case* tersebut. Terakhir, berikan satu situasi di mana rekursi lebih mudah dibaca dibandingkan reduce, dan satu situasi lagi di mana reduce menjadi pilihan yang lebih baik.

CODE ANALYSIS

Tugas Code Analysis terdiri dari Exercise (1 sampai 8) dan Analysis Questions. Selesaikan exercises pada Notebook, dan tulis jawabanmu di *Template laporan yang disediakan.*

Verifikasi & Deadline

- Praktikan **wajib** sudah menyiapkan atau **mengerjakan sebelum pekan materi praktikum** dimulai.
- Praktikan **wajib menunjukkan** pengerjaan Exercise pada Notebook yang sedang berjalan serta laporan praktikum kepada asisten **selama sesi materi berlangsung** untuk mendapatkan penilaian.
- Jika tidak menunjukkan exercise dan laporan kepada asisten, praktikan **tetap wajib mengumpulkan** laporan ke iLab/miLab dalam format **PDF** paling lambat **1 hari setelah praktikum (H+1)**. Namun, **Nilai Laporan Praktikum -40 poin dan mendapat nilai 0 untuk Exercise Implementation**.
- Jika terdapat revisi yang diberikan oleh asisten, baik pada Exercise maupun Explanation Report, maka **praktikan wajib menyelesaikan revisi tersebut selama sesi praktikum berlangsung, tidak ada penerimaan hasil revisi h+1 atau di iLab**

ASSIGNMENT

ASSIGNMENT 1

Kamu bekerja di sebuah hotel dan diberikan data pemesanan kamar berikut:

```
bookings = [
    {"booking_code": "B001", "nights": 3, "price_per_night": 250000, "status": "Confirmed"},
    {"booking_code": "B002", "nights": 2, "price_per_night": 300000, "status": "Pending"},
    {"booking_code": "B003", "nights": 5, "price_per_night": 220000, "status": "Confirmed"},
    {"booking_code": "B004", "nights": 1, "price_per_night": 450000, "status": "Confirmed"},
    {"booking_code": "B005", "nights": 4, "price_per_night": 180000, "status": "Cancelled"},
    {"booking_code": "B006", "nights": 2, "price_per_night": 275000, "status": "Pending"},
    {"booking_code": "B007", "nights": 6, "price_per_night": 200000, "status": "Confirmed"},
    {"booking_code": "B008", "nights": 3, "price_per_night": 320000, "status": "Confirmed"},
    {"booking_code": "B009", "nights": 2, "price_per_night": 260000, "status": "Cancelled"},
    {"booking_code": "B010", "nights": 7, "price_per_night": 190000, "status": "Confirmed"},
    {"booking_code": "B011", "nights": 1, "price_per_night": 400000, "status": "Pending"},
    {"booking_code": "B012", "nights": 4, "price_per_night": 230000, "status": "Confirmed"},
    {"booking_code": "B013", "nights": 5, "price_per_night": 210000, "status": "Confirmed"},
    {"booking_code": "B014", "nights": 2, "price_per_night": 500000, "status": "Cancelled"},
    {"booking_code": "B015", "nights": 3, "price_per_night": 240000, "status": "Confirmed"}
]
```

Selanjutnya, kamu diminta membangun sebuah Hotel Booking Revenue System kecil menggunakan data bookings di atas. Tugas ini menggabungkan semua hal yang telah dipelajari dalam modul ini. Lambda tidak diperbolehkan; setiap fungsi harus diberi nama dan bersifat pure agar tidak mengubah (mutate) list bookings asli.

Lengkapi keempat bagian di bawah ini secara berurutan. Setiap bagian menjelaskan apa yang harus dilakukan oleh fungsi-fungsinya, sementara implementasinya (bagaimana caranya) diserahkan kepadamu. Jalankan setiap cell sebelum melanjutkan ke cell berikutnya, karena tugas-tugas selanjutnya bergantung pada definisi sebelumnya.

Part 1

1. Buat tiga fungsi strategi pricing:
   - standard_price yang mengembalikan total biaya dari masa menginap;
   - discount_price yang mengembalikan total biaya dari masa menginap dengan diskon sebesar 15%;
   - tax_price mengembalikan total biaya masa menginap, termasuk pajak sebesar 10%.
2. Lengkapi apply_pricing() sebagai Higher-Order Function yang menerima sebuah fungsi sebagai parameter dan mengembalikan fungsi baru, yang ketika diterapkan pada satu

data booking, menghasilkan sebuah dictionary booking baru — identik dengan yang asli ditambah satu key ekstra, "total_cost", yang dihitung dengan memanggil strategy pada angka-angka milik booking tersebut.

3. Masukkan fungsi strategi pricing pilihanmu ke dalam Higher-Order Function tersebut, lalu simpan fungsi yang dikembalikan ke dalam variabel bernama pricing_strategy. Kita akan menggunakannya sebagai argumen transformasi pada map() nanti.

Part 2

Terdapat tiga fungsi yang dibutuhkan untuk bagian selanjutnya, masing-masing siap dipakai di filter():

- is_eligible(booking) → bernilai True jika booking tersebut berhak dapat diskon loyalitas: menginap minimal 3 malam, dan statusnya "Confirmed", dua-duanya harus terpenuhi sekaligus.
- is_confirmed(booking) → bernilai True jika status booking-nya "Confirmed" (akan digunakan untuk memisahkan revenue yang masuk).
- is_not_cancelled(booking) → bernilai True untuk booking yang statusnya bukan "Cancelled" (digunakan agar booking yang dibatalkan tidak terhitung ke total malam).

is_not_cancelled sudah disediakan pada kedua blok di bawah ini. Lengkapi is_eligible dan is_confirmed.

Part 3

1. Buat fungsi add yang bersih, yang menerima dua nilai numerik dan mengembalikan total keduanya. Kita akan menggunakan fungsi ini sebagai fungsi penggerak (driving function) untuk reduce() dalam menghitung total agregat pada dataset kita.
2. Kemudian, buat fungsi rekursif (tanpa perulangan atau sum()) yang menjumlahkan field yang diminta (bisa nights atau prices) pada seluruh list datas. Pikirkan baik-baik apa yang harus terjadi ketika datanya kosong (*base case*-mu), dibandingkan dengan ketika masih ada data yang tersisa (*recursive case*-mu).

Part 4

Sekarang apply_pricing, predikat filter, add, dan total_nights sudah siap. Saatnya kamu gabungin semua jadi satu *pipeline*. Gunakan fungsi-fungsi dari Part 1–3. Kamu akan meng-compose fungsi-fungsi untuk menjawab pertanyaan bisnis terkait data, misalnya memperkaya data, mengelompokkan berdasarkan *eligibility*, menghitung total, dll. Tanpa mengubah data bookings yang asli.

Bagian ini dikerjakan langsung (*live*) selama sesi demo. Asisten akan memberikan 2 skenario dan tiap skenario membutuhkan cara *compose* yang berbeda dari fungsi-fungsi Part 1–3. Sebagai contoh, implementasi skenario "total malam dari data bookings yang tidak dibatalkan":

```
not_cancelled = list(filter(is_not_cancelled, bookings))
total_nights_result = total_nights(not_cancelled)
```

```
print("Total nights from non-cancelled bookings")
print(total_nights_result)
```

```
Total nights from non-cancelled bookings
42
```

Kamu harus siap menerapkan pola yang serupa saat *live*, memanggil fungsi yang tepat dari Part 1–3, lalu menggabungkannya (*compose*) menggunakan map/filter/reduce.

Silakan akses MODUL 3-FUNGSIO-26.ipynb untuk template kodenya.

ASSIGNMENT 2

Melanjutkan proyek dari Modul 1 dan 2, sekarang kamu diminta mengembangkan program dengan menerapkan konsep-konsep yang dipelajari di modul ini: **First-Class Functions**, **Higher-Order Functions**, teknik pemrosesan data sequence (**map()**, **filter()**, **reduce()**), dan **Recursion**.

Ketentuan Tugas:

1. Tambahkan fitur baru atau modifikasi fitur yang sudah ada. Implementasi harus memanfaatkan konsep *First-Class Function* (misalnya *higher-order function*) dan Recursion.
2. Minimal terdapat 2 teknik pemrosesan data yang diimplementasikan dari materi modul ini: map(), filter(), dan reduce().
3. Jelaskan secara singkat alasan penggunaan konsep atau fungsi yang dipilih (contoh: mengapa kamu menggunakan filter() untuk memproses data tertentu, atau mengapa kamu merancang fungsi yang menerima fungsi lain sebagai parameter).

ASSESSMENT RUBRIC

Assessment Aspect	Points (Total 100%)
Code Analysis	**10**
Exercise Implementation	5
Practicum Report	5
Assignment 1	**45**
Code / Feature Completeness	15
Code Correctness & Output Accuracy (Live Code)	15
Understanding	15
Assignment 2	**45**
First-Class Function Features	15
Concept Implementation in Data Transformation	15
Understanding	15

DETAILED ASSESSMENT RUBRIC

Code Analysis	Points (Total 10%)
Exercise Implementation	**5**
Seluruh exercise selesai dikerjakan dengan benar (poin akumulasi dari seluruh exercise).	100
Exercise dikerjakan, namun screenshot kode/output salah	-5 (setiap exercise)
Jawaban tidak mengandung identitas praktikan (Nama/NIM)	-2 (setiap exercise)
Selesai melebihi jam praktikum	0 (setiap exercise)

Practicum Report	**5**
Menggunakan template resmi, seluruh bagian *explanation* diisi dengan jawaban yang benar dan jelas	100
Jawaban salah atau jawaban kurang tepat	-5 (setiap exercise)
Penjelasan jawaban ditemukan identik dengan praktikan lain	0 (setiap exercise)
Report dikirim setelah praktikum (overtime)	-40 (dari nilai akhir)
Tidak mengumpulkan	0
Assignment 1	**Points (Total 45%)**
Code / Feature Completeness	**15**
Semua fitur diimplementasikan dengan benar (Part 1–3, tanpa lambda, tanpa mutation).	100
Semua fitur telah diimplementasikan, tetapi terdapat error logika atau format yang kecil.	80
Hanya 2 fitur yang diimplementasikan dengan benar dan fitur lainnya tidak memenuhi sebagian besar ketentuan.	60
Hanya 1 fitur yang diimplementasikan dengan benar, sedangkan fitur lainnya tidak memenuhi sebagian besar ketentuan.	40
Gagal mengimplementasikan atau melanggar ketentuan tugas (Part 1–3).	0
Code Correctness & Accuracy (Live Code)	**15**
Kedua skenario live diselesaikan dengan benar.	100
Keduanya berhasil dikerjakan, namun terdapat bug kecil pada salah satunya.	80
Hanya 1 dari 2 skenario yang benar.	60
Dikerjakan, tetapi muncul kode error/tidak bisa dijalankan.	40
Tidak mengerjakan sama sekali.	0

Understanding	**15**
Menjelaskan dengan detail bagaimana *first-class functions* (1), *higher-order functions* (2), *map/filter/reduce* (4), dan *recursion* (4) diimplementasikan di sepanjang *pipeline*.	100
Mampu menjelaskan sebagian besar konsep (minimal 3), tetapi satu konsep tidak dibahas secara cukup mendalam.	80
Memahami konsep materi secara umum, namun penjelasannya kurang detail.	60
Kesulitan menjelaskan sebagian besar kode dan konsepnya.	40
Tidak dapat menjelaskan implementasi maupun konsepnya sama sekali.	0
Assignment 2	**Points (Total 45%)**
First-Class Function Features	**10**
Mengimplementasikan *Higher-Order Function* dan *Recursion* dengan benar.	100
Mengimplementasikan keduanya, sebagian besar benar, tetapi terdapat sedikit error logika atau kesalahan format.	80
Hanya 1 fitur yang diimplementasikan dengan benar, sedangkan fitur lainnya tidak memenuhi sebagian besar ketentuan.	60
Mengimplementasikan salah satu atau keduanya, tetapi hampir tidak berfungsi atau hanya diimplementasikan secara dangkal.	40
Tidak ada implementasi *Higher-Order Function* dan *Recursion* sama sekali.	0
Concept Implementation in Data Transformation	**10**
Mengimplementasikan dengan benar setidaknya dua dari map()/filter()/reduce(), tanpa mengubah (*mutate*) data asli.	100
Diterapkan dan sebagian besar benar, tetapi terdapat error logika/format kecil.	80
Hanya satu yang diterapkan, atau terdapat error logika yang signifikan/integrasi ke *pipeline* belum lengkap.	60
Dikerjakan tetapi gagal dijalankan atau menghasilkan output yang	40

salah.	
Tidak ada penggunaan `map()`, `filter()`, atau `reduce()` sama sekali.	0
Understanding	**15**
Mampu menjelaskan secara detail alasan penggunaan konsep yang dipilih yang terkait langsung dengan implementasinya.	100
Mampu menjelaskan sebagian besar konsep, tetapi satu konsep tidak dibahas secara cukup mendalam.	80
Hanya mampu menjelaskan sebagian konsep.	60
Kesulitan menjelaskan sebagian besar konsep.	40
Tidak dapat menjelaskan konsepnya sama sekali.	0